6-2장 정리

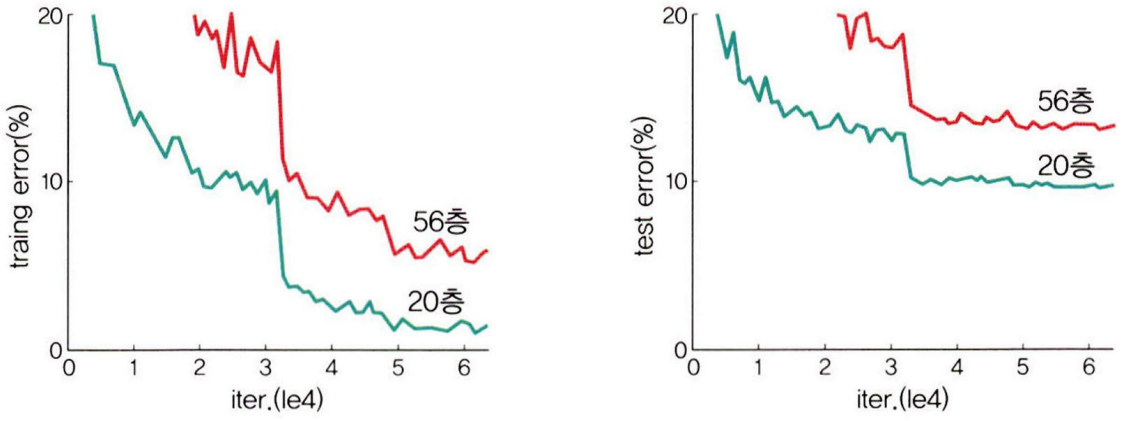
▼ 6.1.5 ResNet

ResNet은 마이크로소프트에서 개발한 알고리즘으로 “Deep Residual Learning for Image Recognition”이라는 논문에서 발표되었음. ResNet 핵심은 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 레지듀얼(residual) 개념을 고안한 것임.

일반적으로 신경망 깊이가 깊어질수록 딥러닝 성능은 좋아질 것 같지만, 실상은 그렇지 않음. “Deep Residual Learning for Image Recognition” 논문에 따르면, 신경망은 깊이가 깊어질수록 성능이 좋아지다가 일정한 단계에 다다르면 오히려 성능이 나빠진다고 함.

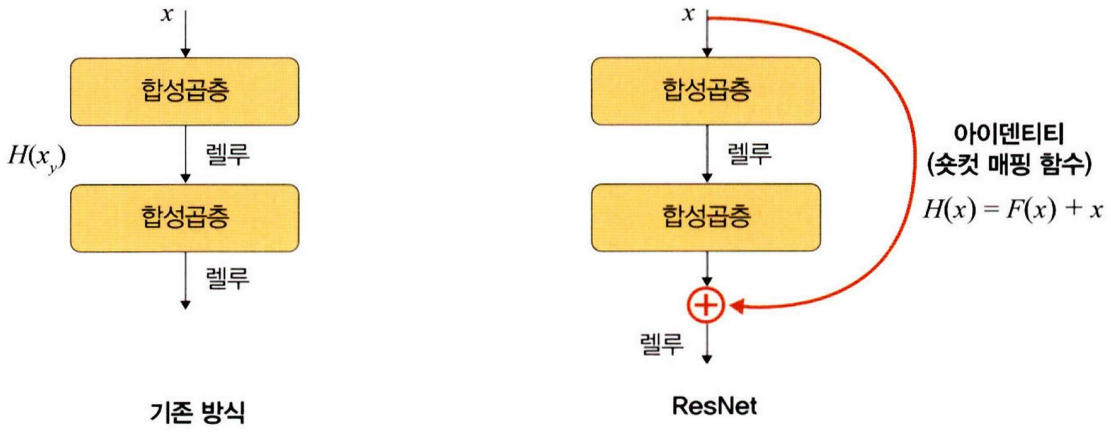
다음 그림과 같이 네트워크 56층이 20층보다 더 나쁜 성능을 보임을 알 수 있음. 즉, 네트워크 깊이가 깊다고 해서 무조건 성능이 좋아지지는 않는다는 것을 보여 주고 있음. ResNet은 바로 이러한 문제를 해결하기 위해 레지듀얼 블록(residual block)을 도입했음. 레지듀얼 블록은 기울기가 잘 전파될 수 있도록 일종의 숏컷(shortcut, skip connection)을 만들어 줌.

❮ 그림 6-24 네트워크 56층이 20층보다 더 나쁜 성능을 보임



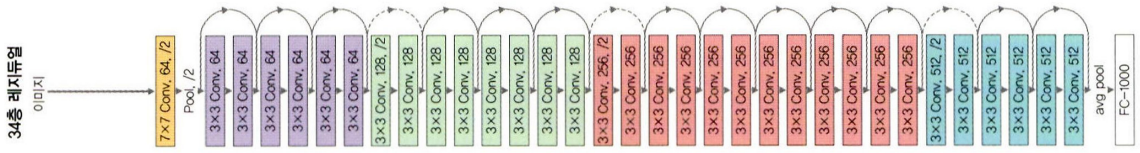
이러한 개념이 필요한 이유는 2014년에 공개된 GoogLeNet은 층이 총 22개로 구성된 것에 비해 ResNet은 층이 총 152개로 구성되어 기울기 소멸 문제가 발생할 수 있기 때문임. 따라서 다음 그림과 같이 숏컷을 두어 기울기 소멸 문제를 방지했다고 이해하면 됨.

❯ 그림 6-25 ResNet 구조



아직까지는 ResNet을 이해하기 어렵음. 차근차근 하나씩 살펴보겠음. 먼저 블록(block)이라는 개념에 대해 알아보겠음. 블록은 계층의 묶음임. 엄밀히 말해서 합성곱층을 하나의 블록으로 묶은 것임. 그림 6-26에서 색상별(보라색, 노란색 등)로 블록을 구분했는데 이렇게 묶인 계층들을 하나의 레지듀얼 블록(residual block)이라고 함. 그리고 레지듀얼 블록을 여러 개 쌓은 것을 ResNet이라고 함.

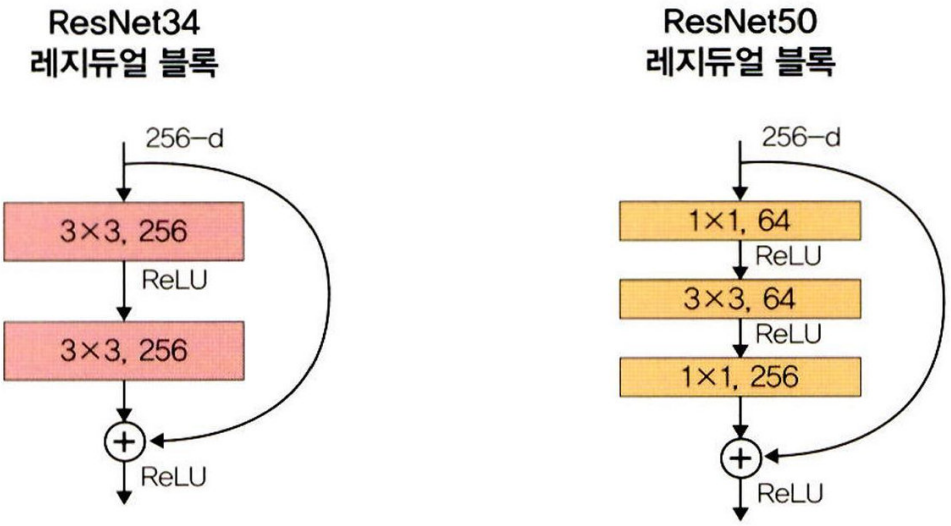
❯ 그림 6-26 ResNet 모델 전체 네트워크



하지만 이렇게 계층을 계속해서 쌓아 늘리면 파라미터 수가 문제가 됨. 계층이 깊어질수록 파라미터는 증가함. 예를 들어 ResNet34는 합성곱층이 34개와 16개의 블록으로 구성되어 있음. 첫 번째 블록의 파라미터가 1152K라면 전체 파라미터 수는 2만 1282K임. 이와 같이 계층의 깊이가 깊어질수록 파라미터는 무제한으로 커질 것임. 이러한 문제를 해결하기 위해 병목 블록(bottleneck block)이라는 것을 두었음.

병목 블록을 두었을 때 어떤 현상이 발생할까요? 다음 그림은 ResNet34와 ResNet50임. ResNet34는 기본 블록(basic block)을 사용하며, ResNet50은 병목 블록을 사용함. 기본 블록의 경우 파라미터 수가 39.3216M인 반면, 병목 블록의 경우 파라미터 수가 6.9632M임. 깊이가 깊어졌음에도 파라미터 수는 감소한 것임.

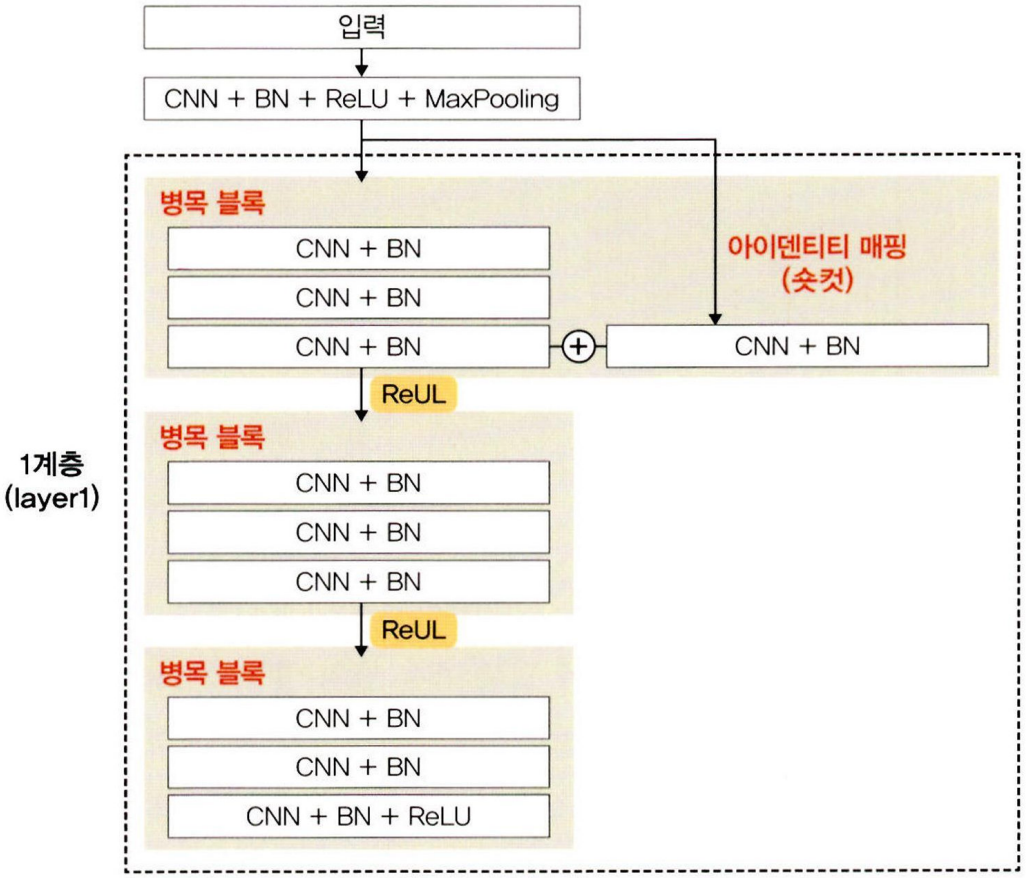
❯ 그림 6-27 기본 블록과 병목 블록



어떻게 가능한 것일까요? 앞에서 분명 깊이가 깊어질수록 파라미터 수가 증가한다고 했음. 하지만 병목 블록을 사용하면 파라미터 수가 감소하는 효과를 줄 수 있음. 합성곱층을 자세히 보면 ResNet34와는 다르게 ResNet50에서는 3×3 합성곱층 앞뒤로 1×1 합성곱층이 붙어 있는데, 1×1 합성곱층의 채널 수를 조절하면서 자원을 줄였다 늘리는 것이 가능하기 때문에 파라미터 수를 줄일 수 있었던 것임. 그리고 이 부분이 병목과 같다고 하여 병목 블록이라고 함.

이제 중요한 아이덴티티 매핑(identity mapping)(혹은 숏컷(shortcut), 스킵 연결(skip connection)이라고도 함)에 대해 알아보겠음. 그림 6-27의 아래쪽에 + 기호가 있음(기본 블록과 병목 블록 모두에서 사용됨). 이 부분을 아이덴티티 매핑이라고 함. 아이덴티티 매핑이란 입력 x가 어떤 함수를 통과하더라도 다시 x라는 형태로 출력되도록 함.

❯ 그림 6-28 아이덴티티 매핑(숏컷)



뭔가 어렵게 느껴지는데 앞으로 구현할 코드를 살펴보면 크게 어렵지 않음. 다음은 앞으로 다룰 코드임. 그중 forward() 부분에 대해서만 살펴보겠음.

```
def forward(self, x):
        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)

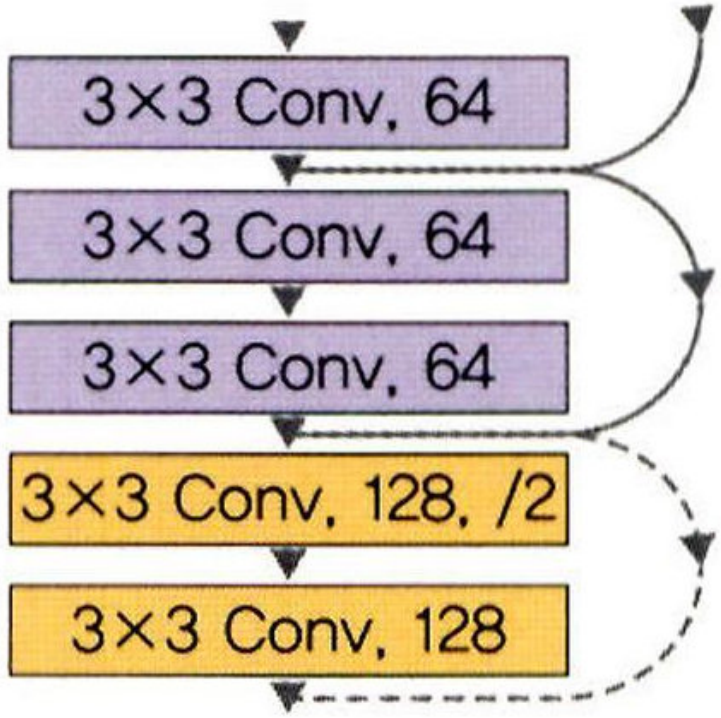
        if self.downsample is not None:
            i = self.downsample(i) ------ 다운샘플 적용

        x += I ------ 아이덴티티 매핑 적용
        x = self.relu(x)
        return x
```

코드를 살펴보면 입력 x를 i라는 변수에 저장했음. 입력 x는 합성곱층을 통과하다가 마지막 x에 i를 더해 주었음. 예를 들어 x가 (28, 28, 64)라고 가정해 봅시다. x를 i 변수에 저장했기 때문에 (28, 28, 64)가 될 것임. 그리고 합성곱층을 통과하면서 같은 형태를 더하기 때문에 최종 형태는 (28, 28, 64) 그대로가 될 것임.

이번에는 또 다른 핵심 개념인 다운샘플(downsample)에 대해 알아보겠음. 다운샘플은 특성 맵(feature map) 크기를 줄이기 위한 것으로 풀링과 같은 역할을 한다고 이해하면 됨. 다음 그림은 ResNet 네트워크의 일부를 가져온 것임.

그림 6-29 ResNet 네트워크의 일부

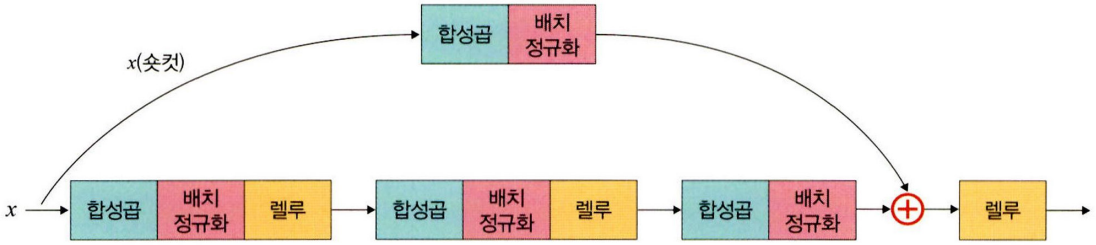


보라색 영역의 첫 번째 블록에서 특성 맵의 형상이 (28, 28, 64)였다면 세 번째 블록의 마지막 합성곱층을 통과하고 아이덴티티 매핑(identity mapping)까지 완료된 특성 맵의 형상도 (28, 28, 64)임. 이번에는 노란색 영역을 살펴볼까요? 노란색 영역의 시작 지점에서는 채널 수가 128로 늘어났고, /2라는 것으로 보아 첫 번째 블록에서 합성곱층의 스트라이드가 2로 늘어나 (14, 14, 128)로 바뀐다는 것을 알 수 있음.

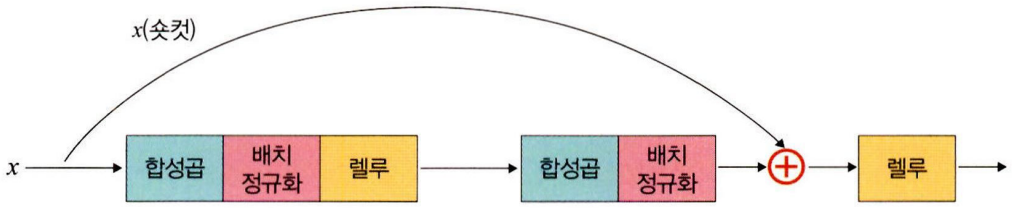
즉, 보라색과 노란색의 형태가 다른데 이들 간의 형태를 맞추지 않으면 아이덴티티 매핑을 할 수 없게 됨. 그래서 아이덴티티에 대해 다운샘플이 필요함.

참고로 입력과 출력의 형태를 같도록 맞추어 주기 위해서는 스트라이드(stride) 2를 가진 1×1 합성곱 계층을 하나 연결해 주면 됨. 이와 같이 입력과 출력의 차원이 같은 것을 아이덴티티 블록이라고 하며, 입력 및 출력 차원이 동일하지 않고 입력의 차원을 출력에 맞추어 변경해야 하는 것을 프로젝션 숏컷(projection-shortcut) 혹은 합성곱 블록이라고 함.

❮ 그림 6-30 합성곱 블록

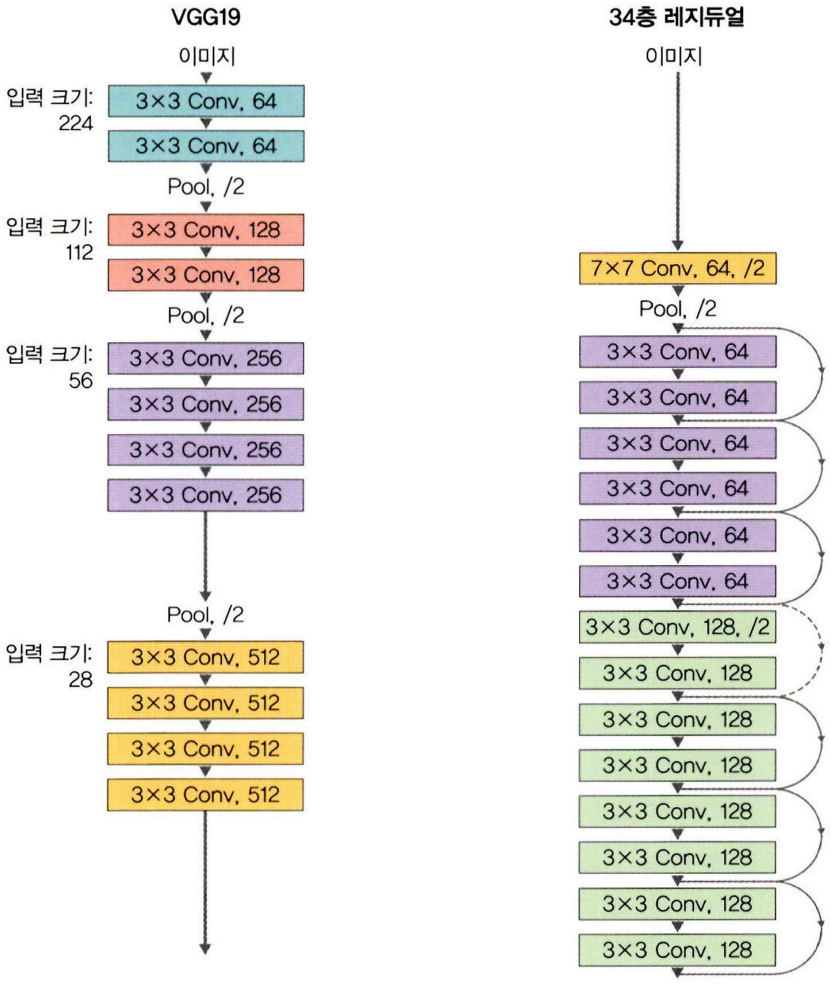


❮ 그림 6-31 아이덴티티 블록



정리하면 ResNet은 기본적으로 VGG19 구조를 뼈대로 하며, 거기에 합성곱층들을 추가해서 깊게 만든 후 숏컷들을 추가하는 것이 사실상 전부라고 생각하면 됨.

❤ 그림 6-32 VGG19와 ResNet 비교



그럼 이제 파이토치로 ResNet을 구현해 보겠음. 이번 예제는 앞에서 살펴보았던 LeNet, AlexNet 등과는 다르게 상당히 복잡하기 때문에 네트워크 구성과 관련하여 주의 깊게 살펴보기 바람.

언제나 그렇듯 필요한 라이브러리를 호출함.

코드 6-65 필요한 라이브러리 호출

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.models as models

import matplotlib.pyplot as plt
import numpy as np

import copy
from collections import namedtuple ------①
import os
import random
import time

import cv2
from torch.utils.data import DataLoader, Dataset
from PIL import Image

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

① 네임드튜플(namedtuple)은 파이썬의 자료형 중 하나임. 말 그대로 튜플의 성질을 갖고 있는 자료형이지만 인덱스뿐만 아니라 키 값으로 데이터에 접근할 수 있음. 따라서 다음과 같이 사용 가능함.

```
from collections import namedtuple
Student = namedtuple('Student', ['name','age','DOB']) ------ 네임드튜플 정의
S = Student('홍길동', '19', '187') ------ 네임드튜플에 값을 추가

print("The Student age using index is : ", end="")
print(S[1]) ------ 인덱스를 이용한 데이터 접근

print("The Student name using keyname is : ", end="")
print(S.name) ------ 키 값을 이용한 데이터 접근
```

네임드튜플에 대한 출력 결과는 다음과 같음.

```
The Student age using index is : 19
The Student name using keyname is : 홍길동
```

출력 결과처럼 인덱스뿐만 아니라 키 값을 이용해도 데이터에 접근할 수 있음을 확인할 수 있음.

다음은 이미지 데이터 전처리에 대한 코드임. 각각의 파라미터는 앞에서 충분히 살펴보았기 때문에 설명은 생략함.

코드 6-66 이미지 데이터 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]), ------ 훈련 이미지 데이터에 대한 전처리
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]) ------ 검증과 테스트 이미지 데이터에 대한 전처리
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

이제 데이터 전처리에서 사용할 변수에 대한 값을 정의함.

코드 6-67 변수에 대한 값 정의

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

torchvision.datasets.ImageFolder를 이용하여 훈련과 테스트 데이터셋을 불러옴. 데이터셋은 개와 고양이에 대한 예제를 계속 사용함. 추가 학습을 위해 캐글에서 이미지 데

이터셋을 내려받아 학습해도 좋음. 혹은 이미지를 찾기 어렵다면 책에서 사용하는 이미지 데이터셋을 이용해도 좋음.

코드 6-68 훈련과 테스트 데이터셋 불러오기

```
cat_directory = r'../chap06/data/dogs-vs-cats/Cat/'
dog_directory = r'../chap06/data/dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_
                               directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.listdir(dog_
                               directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]
```

앞에서 불러온 데이터셋을 훈련, 검증, 테스트 용도로 분리함.

코드 6-69 데이터셋을 훈련, 검증, 테스트 용도로 분리

```
random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400]
val_images_filepaths = correct_images_filepaths[400:-10]
test_images_filepaths = correct_images_filepaths[-10:]
print(len(train_images_filepaths), len(val_images_filepaths),
      len(test_images_filepaths))
```

훈련, 검증, 테스트 데이터셋에 대한 이미지 수는 다음과 같음.

```
400 92 10
```

훈련 데이터셋은 400개, 검증 데이터셋은 92개, 테스트 데이터셋은 열 개로 구성되어 있음.

데이터를 가져와서 전처리를 적용함. 또한, 가져온 데이터가 개이면 레이블 '1'을 부여하고, 고양이라면 레이블 '0'을 부여함.

코드 6-70 이미지에 대한 레이블 구분

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):



        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path)
        img_transformed = self.transform(img, self.phase)

        label = img_path.split('/')[-1].split('.')[0]
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0
        return img_transformed, label
```

훈련과 검증 용도의 데이터셋을 정의함.

코드 6-71 이미지 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size,
                                mean, std), phase='train')
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size,
                              mean, std), phase='val')

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])
```

다음은 훈련 데이터셋 index 0의 이미지 크기와 레이블에 대한 출력 결과임.

```
torch.Size([3, 224, 224])
0
```

이미지는 컬러(채널 3) 상태에서 224×224 크기를 갖고 있으며 레이블이 0이므로 고양이를 의미함.

데이터로더를 이용하여 데이터를 메모리로 불러옴. 불러올 때는 배치 크기만큼 나누어서 불러옴.

코드 6-72 데이터셋의 데이터를 메모리로 불러오기

```
train_iterator = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_iterator = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_iterator, 'val': valid_iterator}

batch_iterator = iter(train_iterator)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

다음은 데이터로더를 이용하여 메모리로 불러온 훈련 데이터셋의 이미지 크기와 레이블에 대한 출력 결과임.

```
torch.Size([32, 3, 224, 224])
tensor([0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1,
0, 1, 0, 1, 0, 1])
```

이제 ResNet의 전체 네트워크 구성을 위해 그것을 구성하는 기본 블록과 병목 블록에 대한 코드를 먼저 살펴보겠음. 먼저 기본 블록은 ResNet18, ResNet34에서 사용되며 합성곱(3×3) 두 개로 구성됨.

코드 6-73 기본 블록 정의

```
class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                               stride=stride, padding=1, bias=False) ------ 3×3 합성곱층
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                               stride=1, padding=1, bias=False) ------ 3×3 합성곱층
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample: ------ ①
            conv = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                             stride=stride, bias=False)
            bn = nn.BatchNorm2d(out_channels)
            downsample = nn.Sequential(conv, bn)



        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)

        if self.downsample is not None:
            i = self.downsample(i)

        x += I ------ ②
        x = self.relu(x)

        return x
```

① 다운샘플(downsample)이 적용되는 부분임. 다운샘플은 입력 데이터의 크기와 네트워크를 통과한 후 출력 데이터의 크기가 다를 경우에 사용함. 다운샘플을 위해서는 다음과 같이 합성곱층에 스트라이드를 적용함.

```
conv = nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=1, bias=False)
```

(예제에서는 stride=stride로 되어 있지만, 클래스 속성으로 stride=1이 정의되었기 때문에 stride=1로 작성했음)

② 아이덴티티 매핑이 적용되는 부분임. 특정 층에 존재하는 출력 결과를 다음 합성곱층을 통과한 출력 결과에 더해 준다고 하여 스킵 연결(skip connection)이라고도 함. 예를 들어 다음과 같이 x에 conv1, bn1, relu, conv2, bn2, relu, conv3, bn3 값이 더해지다가 초기의 x가 다시 더해지는 것을 아이덴티티 매핑 혹은 숏컷 혹은 스킵 연결이라고 함.

```
def forward(self, x):
        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)



    x = self.bn2(x)
    x = self.relu(x)
    x = self.conv3(x)
    x = self.bn3(x)

    if self.downsample is not None:
        i = self.downsample(i)

    x += i
    x = self.relu(x)
    return x
```

병목 블록은 ResNet50, ResNet101, ResNet152에서 사용되며 1×1 합성곱층, 3×3 합성곱층, 1×1 합성곱층으로 구성됨.

코드 6-74 병목 블록 정의

```
class Bottleneck(nn.Module):
    expansion = 4 ------ ResNet에서 병목 블록을 정의하기 위한 하이퍼파라미터입니다.

    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                               stride=1, bias=False) ------ 1×1 합성곱층
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                               stride=stride, padding=1, bias=False) ------ 3×3 합성곱층
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv3 = nn.Conv2d(out_channels, self.expansion*out_channels, ------
                               kernel_size=1, stride=1, bias=False)
        self.bn3 = nn.BatchNorm2d(self.expansion*out_channels)
        self.relu = nn.ReLU(inplace=True)
                                                  1×1 합성곱층, 또한 다음 계층의 입력 채널 수와
                                                일치하도록 self.expansion*out_channels를 합니다.
        if downsample:
            conv = nn.Conv2d(in_channels, self.expansion*out_channels, kernel_size=1,
                             stride=stride, bias=False)
            bn = nn.BatchNorm2d(self.expansion*out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):



        i = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu(x)
        x = self.conv3(x)
        x = self.bn3(x)

        if self.downsample is not None:
            i = self.downsample(i)

        x += i
        x = self.relu(x)
        return x
```

병목 블록에 대한 코드를 살펴보았는데 기본 블록과 병목 블록에 대한 차이가 있었음. 기본 블록이 3×3 합성곱층 두 개를 갖는 반면, 병목 블록은 1×1 합성곱층, 3×3 합성곱층, 1×1 합성곱층의 구조를 갖음. 기본 블록을 병목 블록으로 변경하는 이유는 계층을 더 깊게 쌓으면서 계산에 대한 비용을 줄일 수 있기 때문임. 그리고 계층이 많아진다는 것은 곧 활성화 함수가 기존보다 더 많이 포함된다는 것이고, 이것은 더 많은 비선형성(non-linearity)을 처리할 수 있음을 의미하기도 함. 즉, 다양한 입력 데이터에 대한 처리가 가능하다는 의미임.

결국 아이덴티티 매핑과 병목 블록으로 ResNet 네트워크에 더욱 깊은 계층을 쌓을 수 있게 된 것임.

이제 ResNet 모델에 대한 네트워크를 정의해 봅시다. ResNet 역시 VGG처럼 다양한 모델이 있음.

코드 6-75 ResNet 모델 네트워크

```
class ResNet(nn.Module):
    def __init__(self, config, output_dim, zero_init_residual=False):
        super().__init__()

        block, n_blocks, channels = config ---- ResNet을 호출할 때 넘겨준 config 값들을
                                                block, n_blocks, channels에 저장
        self.in_channels = channels[0]
        assert len(n_blocks) == len(channels) == 4 ------ 블록 크기=채널 크기=4
```

```
        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size=7, stride=2,
                               padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        self.layer1 = self.get_resnet_layer(block, n_blocks[0], channels[0])
        self.layer2 = self.get_resnet_layer(block, n_blocks[1], channels[1], stride=2)
        self.layer3 = self.get_resnet_layer(block, n_blocks[2], channels[2], stride=2)
        self.layer4 = self.get_resnet_layer(block, n_blocks[3], channels[3], stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1,1))
        self.fc = nn.Linear(self.in_channels, output_dim)

        if zero_init_residual: ------ ①
            for m in self.modules():
                if isinstance(m, Bottleneck):
                    nn.init.constant_(m.bn3.weight, 0)
                elif isinstance(m, BasicBlock):
                    nn.init.constant_(m.bn2.weight, 0)

    def get_resnet_layer(self, block, n_blocks, channels, stride=1): ------ 블록을 추가하기 위한 함수
        layers = []
        if self.in_channels != block.expansion * channels:
            downsample = True ------ in_channels와 block.expansion*channels가 다르면 downsample 적용
        else:
            downsample = False
        layers.append(block(self.in_channels, channels, stride, downsample)) ------ 계층(layer)을 추가할 때 in_channels, channels, stride뿐만 아니라 다운샘플 적용 유무도 함께 전달
        for i in range(1, n_blocks): ------ n_blocks만큼 계층 추가
            layers.append(block(block.expansion*channels, channels))

        self.in_channels = block.expansion * channels
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.conv1(x) ------ 224 × 224
        x = self.bn1(x)
        x = self.relu(x)
        x = self.maxpool(x) ------ 112 × 112
        x = self.layer1(x) ------ 56 × 56
        x = self.layer2(x) ------ 28 × 28
        x = self.layer3(x) ------ 14 × 14
```

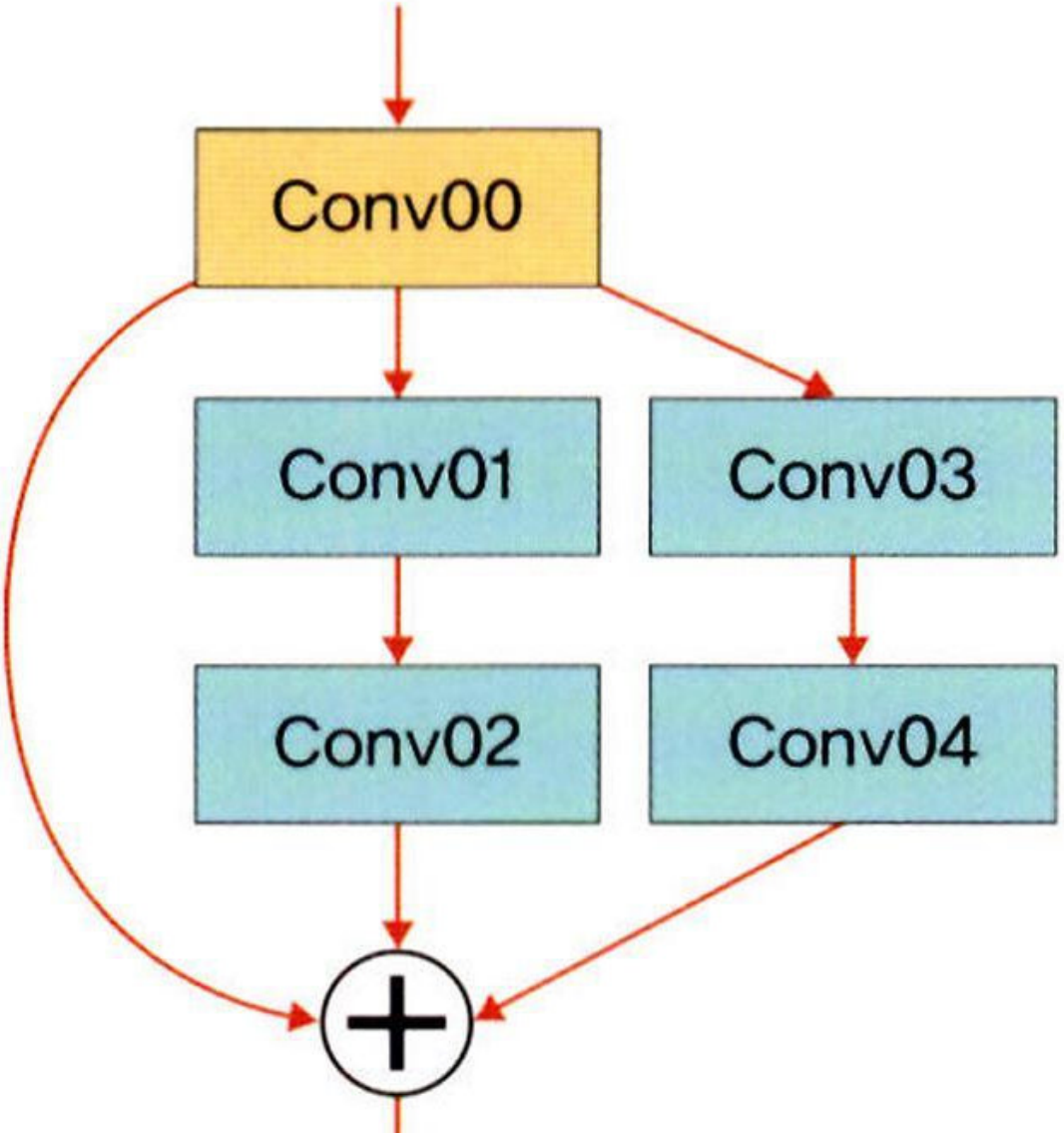
```
x = self.layer4(x) ------ 7×7
x = self.avgpool(x) ------ 1×1
h = x.view(x.shape[0], -1)
x = self.fc(h)
return x, h
```

① 각 레지듀얼 분기(residual branch)에 있는 마지막 BN(Batch Normalization)을 0으로 초기화해서 다음 레지듀얼 분기를 0에서 시작할 수 있도록 함. 이 부분은 모델을 생성하고 학습시키는 것과는 상관없지만, 다음 URL 논문에 의하면 BN을 0으로 초기화할 경우 모델 성능이 0.2~0.3% 정도 향상된다고 함. 따라서 ResNet에서는 많이 사용되고 있음.

https://arxiv.org/abs/1706.02677

참고로 레지듀얼 분기란 프로그램에서의 조건에 따라 A, B, C 등으로 분기하는 것과 같음.

❯ 그림 6-33 레지듀얼 분기



ResNetConfig 변수에 네임드튜플 데이터 형식으로 ['block', 'n_blocks', 'channels']를 저장함.

코드 6-76 ResNetConfig 정의

```
ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_blocks', 'channels'])
```

먼저 기본 블록을 사용하는 ResNet18과 ResNet34의 Config를 정의함. 즉, 블록은 기본 블록(BasicBlock)을 사용하도록 하며, 블록(n_blocks)과 채널(channels)의 크기를 각각 지정함.

코드 6-77 기본 블록을 사용하여 ResNetConfig 정의

```
resnet18_config = ResNetConfig(block=BasicBlock,
                               n_blocks=[2,2,2,2],
                               channels=[64,128,256,512])

resnet34_config = ResNetConfig(block=BasicBlock,
                               n_blocks=[3,4,6,3],
                               channels=[64,128,256,512])
```

이번에는 병목 블록을 사용하는 ResNet50, ResNet101, ResNet152의 Config를 정의함. 즉, 블록은 병목 블록(Bottleneck)을 사용하도록 하며, 블록(n_blocks)과 채널(channels)의 크기를 각각 지정함.

코드 6-78 병목 블록을 사용하여 ResNetConfig 정의

```
resnet50_config = ResNetConfig(block=Bottleneck,
                               n_blocks=[3,4,6,3],
                               channels=[64,128,256,512])

resnet101_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,4,23,3],
                                channels=[64,128,256,512])

resnet152_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,8,36,3],
                                channels=[64,128,256,512])
```

기본적인 합성곱층을 사용하는 것도 복잡한데 아이덴티티 매핑, 다운샘플 및 병목 블록까지 정의하여 사용하려니 꽤 번거로움.

ResNet 역시 사전 훈련된 모델이기 때문에 코드 한 줄로 간단하게 사용할 수 있음. 이제 그 방법에 대해 잠깐 살펴보겠음.

코드 6-79 사전 훈련된 ResNet 모델 사용

```
pretrained_model = models.resnet50(pretrained=True)
```

------ 사전 훈련된 ResNet 모델 사용을 위해서는 pretrained=True로 설정

우리가 만든 ResNet 네트워크와 사전 훈련된 ResNet 네트워크가 동일한지 확인해 봅시다.

코드 6-80 사전 훈련된 ResNet 네트워크 확인

```
print(pretrained_model)
```

다음은 사전 훈련된 ResNet 네트워크를 출력한 결과임.

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
    )
```

```
... 중간 생략 ...
  (layer4): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_
```

```
stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=2048, out_features=1000, bias=True)
)
```

직접 작성한 ResNet 네트워크로 다시 돌아오겠음. ResNet50 모델을 사용하기 위해 Config 부분에 resnet50_config를 지정하고 OUTPUT_DIM에는 테스트 데이터셋의 클래스(개와 고양이)를 입력함.

코드 6-81 ResNet50 Config를 사용한 ResNet 모델 사용

```
OUTPUT_DIM = 2 ------ 두 개의 클래스 사용(개와 고양이)
model = ResNet(resnet50_config, OUTPUT_DIM)
print(model)
```

다음은 수동으로 작성한 ResNet 모델의 네트워크 구조임. 앞에서 살펴보았던 사전 훈련된 ResNet 네트워크와 비교해서 살펴보기 바람(네트워크 깊이가 깊기 때문에 책에서는 중간 생략함).

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
```

```
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
    )
... 중간 생략 ...
  (layer4): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
```

```
bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_
stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=2048, out_features=2, bias=True)
)
```

어떤가요? 힘들게 정의한 네트워크와 사전 정의된 네트워크가 다르지 않음. 앞으로 ResNet을 사용할 필요가 있을 경우 사전 훈련된 모델을 코드 한두 줄로 불러와 사용하게 될 것임. 하지만 그 구조를 알고 사용하는 것과 모르고 사용하는 것은 큰 차이가 있음. 추후 네트워크를 변경하거나 입출력 텐서의 크기를 알고자 할 경우 네트워크를 알고 있다면 수정이 어렵지 않을 것임. 따라서 향후 논문에서 새로운 모델의 네트워크가 발표되더라도 그 구조부터 익히는 훈련이 필요함.

모델 학습을 위해 옵티마이저와 손실 함수를 정의함.

코드 6-82 옵티마이저와 손실 함수 정의

```
optimizer = optim.Adam(model.parameters(), lr=1e-7) ------ lr=1e-7은 1*10의 -7승을 의미
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)
```

모델이 얼마나 잘 학습되었는지 측정하기 위한 함수를 정의함.

코드 6-83 모델 학습 정확도 측정 함수 정의

```
def calculate_topk_accuracy(y_pred, y, k=2):
    with torch.no_grad():
        batch_size = y.shape[0]
        _, top_pred = y_pred.topk(k, 1) ------①
        top_pred = top_pred.t() ------②
        correct = top_pred.eq(y.view(1, -1).expand_as(top_pred)) ------③
        correct_1 = correct[:1].reshape(-1).float().sum(0, keepdim=True)
        correct_k = correct[:k].reshape(-1).float().sum(0, keepdim=True)
        acc_1 = correct_1 / batch_size
        acc_k = correct_k / batch_size
    return acc_1, acc_k
```

이미지의 정확한 레이블 부여를 위해 사용. 즉, 첫 번째 레이블이 아닌 정확한 레이블 부여를 위해 사용

① tensor.topk는 torch.argmax와 같은 효과임. 주어진 텐서에서 가장 큰 값의 인덱스(index)를 얻기 위해 사용함. 즉, 네트워크의 출력에서 가장 확률이 높은 값의 인덱스를 반환함. 다음 예시로 사용 방법을 확인할 수 있음.

```
import torch
x = torch.arange(1., 6.)
print(x)
print('-----------------')
print(torch.topk(x, 3)) ------ x 입력에서 가장 큰 값 세 개를 선택하여 그 값과 인덱스를 출력
```

출력 결과는 다음과 같음.

```
tensor([1., 2., 3., 4., 5.])
-----------------
torch.return_types.topk(
values = tensor([5., 4., 3.]),
indices = tensor([4, 3, 2]))
```

② t()는 차원 0과 1을 전치(transpose)하겠다는 의미임. 예를 들어 다음과 같이 사용됨.

```
x = torch.randn(3) ------ 1차원 텐서 생성
print(x)
print(torch.t(x)) ------ 1차원 그대로 값을 반환
print('------------')
x = torch.randn(2, 3) ------ (2, 3)차원 텐서 생성
```

```
print(x)
print(torch.t(x)) ------ 입력이 전치되어 (3, 2)차원 반환
```

코드를 실행하면 다음과 같이 출력됨.

```
tensor(0.7357)
tensor(0.7357)
------------
tensor([[-0.3894,  0.6999, -0.6733],
        [ 1.7850,  0.1961,  0.2701]])
tensor([[-0.3894,  1.7850],
        [ 0.6999,  0.1961],
        [-0.6733,  0.2701]])
```

③ 텐서를 비교하는 함수로, 텐서가 서로 같은지를 비교한다면 torch.eq, 다른지를 비교한다면 torch.ne, 크거나 같은지를 비교한다면 torch.ge를 사용함. torch.eq는 다음과 같은 형식을 이용하여 사용함.

```
torch.eq(비교 대상 텐서, 비교할 텐서)
```

torch.eq의 결과는 각 텐서의 요소들을 비교해서 같으면 True, 다르면 False를 반환함.

예를 들어 다음과 같이 사용함.

```
torch.eq(torch.tensor([[1, 2], [3, 4]]), torch.tensor([[1, 1], [4, 4]]))
```

1과 1을 비교했을 때 서로 같으므로 True, 2와 1을 비교했을 때 서로 다르므로 False, 3과 4를 비교했을 때 서로 다르므로 False, 4와 4를 비교했을 때 서로 같으므로 True를 반환함. 코드를 실행하면 다음과 같이 출력될 것임.

```
tensor([[ True, False],
        [False,  True]])
```

모델을 학습시키는 방법에 대한 함수를 정의함.

코드 6-84 모델 학습 함수 정의

```
def train(model, iterator, optimizer, criterion, scheduler, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0



    model.train()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred = model(x)
        loss = criterion(y_pred[0], y)

        acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc_1 += acc_1.item() ------ 모델이 첫 번째로 예측한 레이블이 붙여집니다.
        epoch_acc_5 += acc_5.item() ------ 이미지에 정확한 레이블이 붙여질 것이기 때문에 정확도가 100%일 것입니다.

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

모델에 대한 평가 함수를 정의함.

코드 6-85 모델 평가 함수 정의

```
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred = model(x)
            loss = criterion(y_pred[0], y)

            acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
            epoch_loss += loss.item()
            epoch_acc_1 += acc_1.item()
            epoch_acc_5 += acc_5.item()



    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

모델을 학습시키는 데 어느 정도의 시간이 걸리는지 측정하기 위한 함수를 정의함.

코드 6-86 모델 학습 시간 측정 함수 정의

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time/60)
    elapsed_secs = int(elapsed_time-(elapsed_mins*60))
    return elapsed_mins, elapsed_secs
```

이제 본격적으로 훈련과 검증 데이터셋을 이용하여 모델을 학습시킴.

코드 6-87 모델 학습

```
best_valid_loss = float('inf')
EPOCHS = 10

for epoch in range(EPOCHS):
    start_time = time.monotonic()

    train_loss, train_acc_1, train_acc_5 = train(model, train_iterator, optimizer,
                                                 criterion, scheduler, device)
    valid_loss, valid_acc_1, valid_acc_5 = evaluate(model, valid_iterator, criterion,
                                                    device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), '../chap06/data/ResNet-model.pt')

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc @1: {train_acc_1*100:6.2f}% | ' \
          f'Train Acc @5: {train_acc_5*100:6.2f}%')
    print(f'\tValid Loss: {valid_loss:.3f} | Valid Acc @1: {valid_acc_1*100:6.2f}% | ' \
          f'Valid Acc @5: {valid_acc_5*100:6.2f}%')
```

다음은 모델 학습에 대한 결과임.

```
Epoch: 01 | Epoch Time: 4m 4s
 Train Loss: 0.698 | Train Acc @1:  50.24% | Train Acc @5: 100.00%
 Valid Loss: 0.699 | Valid Acc @1:  51.19% | Valid Acc @5: 100.00%
Epoch: 02 | Epoch Time: 4m 10s
 Train Loss: 0.693 | Train Acc @1:  50.48% | Train Acc @5: 100.00%
 Valid Loss: 0.699 | Valid Acc @1:  51.19% | Valid Acc @5: 100.00%
Epoch: 03 | Epoch Time: 4m 13s
 Train Loss: 0.701 | Train Acc @1:  49.04% | Train Acc @5: 100.00%
 Valid Loss: 0.696 | Valid Acc @1:  47.17% | Valid Acc @5: 100.00%
Epoch: 04 | Epoch Time: 4m 11s
 Train Loss: 0.696 | Train Acc @1:  49.52% | Train Acc @5: 100.00%
 Valid Loss: 0.698 | Valid Acc @1:  48.96% | Valid Acc @5: 100.00%
Epoch: 05 | Epoch Time: 4m 2s
 Train Loss: 0.695 | Train Acc @1:  49.52% | Train Acc @5: 100.00%
 Valid Loss: 0.700 | Valid Acc @1:  50.00% | Valid Acc @5: 100.00%
Epoch: 06 | Epoch Time: 4m 3s
 Train Loss: 0.686 | Train Acc @1:  58.41% | Train Acc @5: 100.00%
 Valid Loss: 0.693 | Valid Acc @1:  49.85% | Valid Acc @5: 100.00%
Epoch: 07 | Epoch Time: 4m 7s
 Train Loss: 0.686 | Train Acc @1:  52.64% | Train Acc @5: 100.00%
 Valid Loss: 0.690 | Valid Acc @1:  51.93% | Valid Acc @5: 100.00%
Epoch: 08 | Epoch Time: 4m 7s
 Train Loss: 0.688 | Train Acc @1:  57.45% | Train Acc @5: 100.00%
 Valid Loss: 0.693 | Valid Acc @1:  49.55% | Valid Acc @5: 100.00%
Epoch: 09 | Epoch Time: 4m 0s
 Train Loss: 0.690 | Train Acc @1:  55.29% | Train Acc @5: 100.00%
 Valid Loss: 0.692 | Valid Acc @1:  54.32% | Valid Acc @5: 100.00%
Epoch: 10 | Epoch Time: 4m 1s
 Train Loss: 0.690 | Train Acc @1:  57.69% | Train Acc @5: 100.00%
 Valid Loss: 0.693 | Valid Acc @1:  51.93% | Valid Acc @5: 100.00%
```

역시 모델 학습 결과에 대한 오차와 정확도 측면에서 성능이 좋지 않음. 계속 언급하지만 이미지 데이터를 늘린다면 성능은 좋아질 수 있음. 이 예제의 목적은 성능 향상이 아닌 CNN 관련 네트워크의 사용 방법이므로, 빠른 학습을 위해 데이터 개수를 제한시켰기 때문에 성능은 좋지 않음.

모델이 얼마나 잘 학습되었는지 알아보기 위해 테스트 데이터셋을 이용하여 예측 결과를 ResNet.csv로 저장함.

코드 6-88 테스트 데이터셋을 이용한 모델 예측

```
import pandas as pd
id_list = []
pred_list = []
_id = 0
with torch.no_grad():
    for test_path in test_images_filepaths:
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val')
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs[0], dim=1)[:, 1].tolist()
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('../chap06/data/ResNet.csv', index=False)
res.head(10)
```

다음은 테스트 데이터셋의 예측 결과인 ResNet.csv를 출력한 결과임.

	id	label
0	109	0.499019
1	145	0.528704
2	15	0.536826
3	162	0.484544
4	167	0.503171
5	200	0.585335
6	210	0.539800
7	211	0.503852
8	213	0.471151
9	224	0.524336

레이블(label)이 0.5보다 크면 개를 의미하고, 0.5보다 작으면 고양이를 의미함.

이제 모델을 얼마나 잘 예측했는지 이미지를 통해 확인해 봅시다.

코드 6-89 모델 예측에 대한 결과 출력

```
class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]

        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
display_image_grid(test_images_filepaths)
```

다음 그림은 테스트 데이터셋을 이용한 예측 결과를 이미지로 출력한 결과임.

❯ 그림 6-34 테스트 데이터셋에 대한 ResNet 모델의 예측 결과



역시 정확도가 높지 않음. 계속 이야기하지만 성능을 향상시키려면 데이터를 더 늘려야 함. 책에서는 학습 용도로 CNN의 다양한 모델을 어떻게 사용하는지에 집중했음. 즉, 네트워크 위주의 학습을 위해 일부로 동일한 데이터셋을 사용했음. 따라서 이 장에서는 CNN의 다양한 모델의 네트워크 구성 위주로 학습하는 것을 권장함. 성능 향상 관련해서는 8장을 참고하세요.

컴퓨터 비전 분야에서 객체를 분류하는 방법에 대해 감이 좀 오나요? 중요한 것은 모델을 학습하기 위한 데이터임. 신경망(혹은 네트워크)은 이미 구현된 모델을 재사용할 수 있는 것이 많기에 우리는 단지 누군가가 만들어 놓은 신경망을 가져다 쓰기만 하면 됨. 중요한 점은 내가 가진 데이터에 가장 적합한 모델을 선택하는 것임. 이후에는 앞서 배운 전이 학습을 사용하여 약간의 튜닝만 진행하면 됨.

다음 절에서는 객체 인식에 대해 살펴보겠음.

▼ 6.2 객체 인식을 위한 신경망

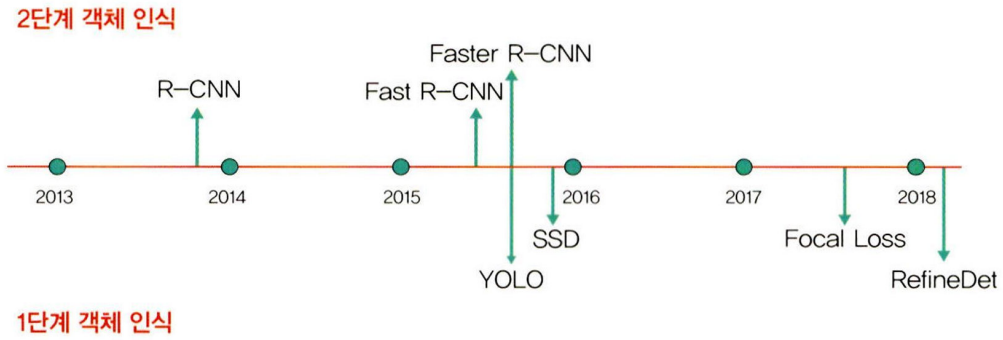
객체 인식(object detection)은 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술임. 즉, 객체 인식이란 이미지나 영상 내에 있는 여러 객체에 대해 각 객체가 무엇인지 분류하는 문제와 그 객체 위치가 어디인지 박스(bounding box)로 나타내는 위치 검출(localization) 문제를 다루는 분야임.

따라서 객체 인식은 다음과 같이 표현할 수 있음.

객체 인식 = 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출

딥러닝을 이용한 객체 인식 알고리즘은 크게 1단계 객체 인식(1-stage detector)과 2단계 객체 인식(2-stage detector)으로 나눌 수 있음.

❯ 그림 6-35 1단계 객체 인식 vs 2단계 객체 인식 흐름도



1단계 객체 인식은 이 두 문제(분류와 위치 검출)를 동시에 행하는 방법이고, 2단계 객체 인식은 이 두 문제를 순차적으로 행하는 방법임. 따라서 1단계 객체 인식은 비교적 빠르지만 정확도가 낮고, 2단계 객체 인식은 비교적 느리지만 정확도가 높음.

2단계 객체 인식은 CNN을 처음으로 적용시킨 R-CNN 계열이 대표적이며, 1단계 객체 인식에는 YOLO(You Only Look Once) 계열과 SSD 계열 등이 포함됨.

참고로 객체 인식은 자율 주행 자동차, CCTV, 무인 점포 등 많은 곳에서 활용함.

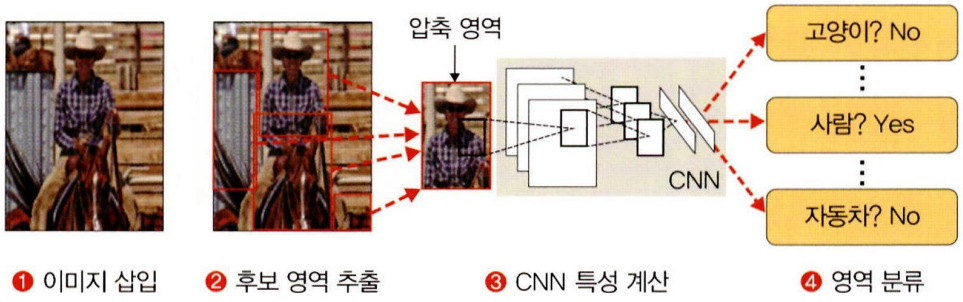
여기에서는 2단계 객체 인식 알고리즘을 알아보겠음.

6.2.1 R-CNN

예전의 객체 인식 알고리즘들은 슬라이딩 윈도우(sliding window) 방식, 즉 일정한 크기를 가지는 윈도우(window)를 가지고 이미지의 모든 영역을 탐색하면서 객체를 검출해 내는 방식이었음. 하지만 알고리즘의 비효율성 때문에 많이 사용하지 않았으며, 현재는 선택적 탐색(selective search) 알고리즘을 적용한 후보 영역(region proposal)을 많이 사용함.

R-CNN(Region-based CNN)은 이미지 분류를 수행하는 CNN과 이미지에서 객체가 있을 만한 영역을 제안해 주는 후보 영역 알고리즘을 결합한 알고리즘임. R-CNN의 수행 과정은 다음 그림과 같음.

그림 6-36 R-CNN 학습 절차



1. 이미지를 입력으로 받음.
2. 2000개의 바운딩 박스(bounding box)를 선택적 탐색 알고리즘으로 추출한 후 잘라 내고(cropping), CNN 모델에 넣기 위해 같은 크기(227×227 픽셀)로 통일함(warping).
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용함.
4. 각각 분류를 진행하여 결과를 도출함.

R-CNN은 성능이 뛰어나기는 하지만 다음과 같은 단점으로 크게 발전하지는 못했음.

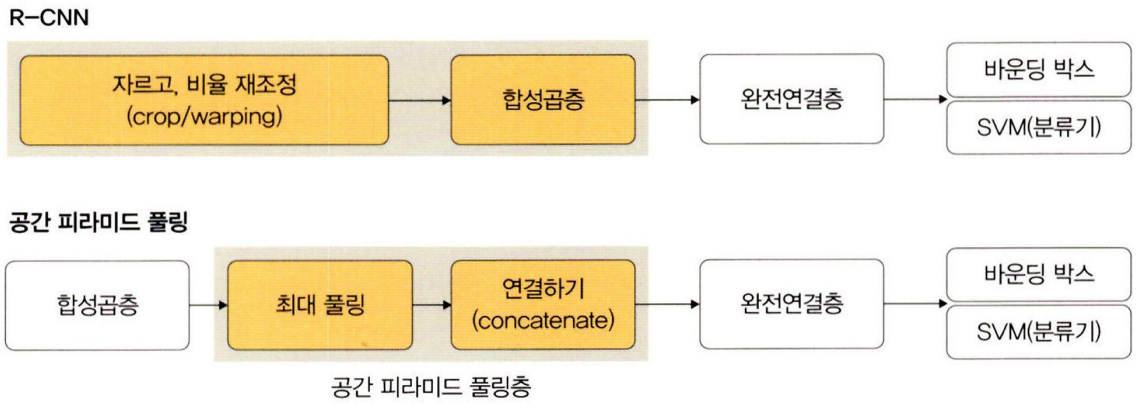
1. 앞서 언급한 세 단계의 복잡한 학습 과정
2. 긴 학습 시간과 대용량 저장 공간
3. 객체 검출(object detection) 속도 문제

이러한 문제를 해결하기 위해 Fast R-CNN이 생겼음.

6.2.2 공간 피라미드 풀링

기존 CNN 구조(예 다음 그림의 R-CNN)들은 모두 완전연결층을 위해 입력 이미지를 고정해야 했음. 그렇기 때문에 신경망을 통과시키려면 이미지를 고정된 크기로 자르거나(crop) 비율을 조정(warp)해야 했음. 하지만 이렇게 하면 물체의 일부분이 잘리거나 본래의 생김새와 달라지는 문제점이 있음. 이러한 문제를 해결하고자 공간 피라미드 풀링(spatial pyramid pooling)을 도입했음.

❯ 그림 6-40 공간 피라미드 풀링



공간 피라미드 풀링층

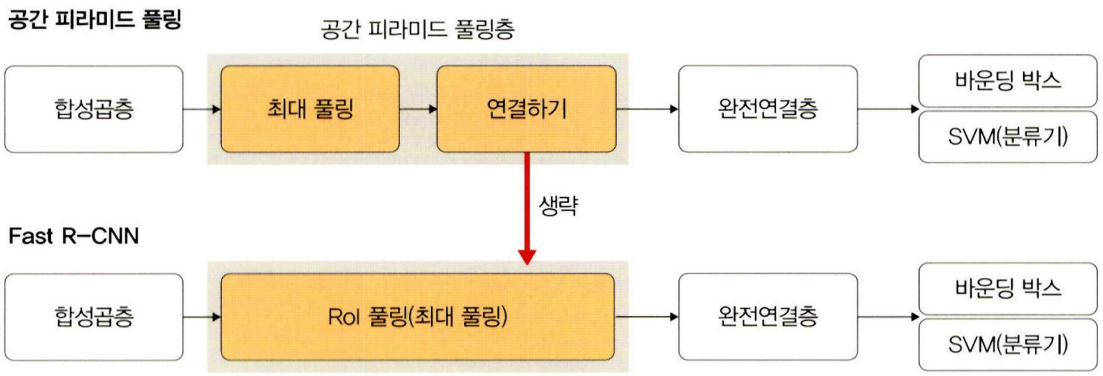
즉, 공간 피라미드 풀링은 입력 이미지의 크기에 관계없이 합성곱층을 통과시키고, 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해 주는 풀링층을 적용하는 기법임.

입력 이미지의 크기를 조절하지 않고 합성곱층을 통과시키기 때문에 원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있음. 또한, 이미지 분류나 객체 인식 같은 여러 작업에 적용할 수 있다는 장점이 있음.

6.2.3 Fast R-CNN

R-CNN은 바운딩 박스마다 CNN을 돌리고, 분류를 위한 긴 학습 시간이 문제였음. Fast R-CNN(Fast Region-based CNN)은 R-CNN의 속도 문제를 개선하려고 RoI 풀링을 도입했음. 즉, 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 하고 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기를 조정함. 이렇게 하면 바운딩 박스마다 CNN을 돌리는 시간을 단축할 수 있음.

❯ 그림 6-41 Fast R-CNN

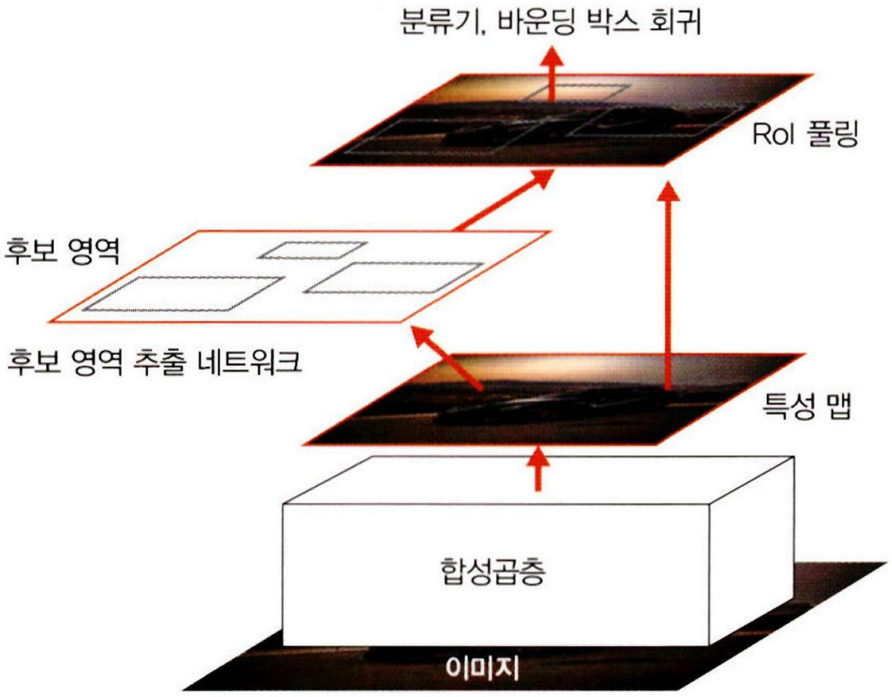


6.2.4 Faster R-CNN

Faster R-CNN은 '더욱 빠른' 객체 인식을 수행하기 위한 네트워크임. 기존 Fast R-CNN 속도의 걸림돌이었던 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계했음. 즉, Faster R-CNN은 기존 Fast R-CNN에 후보 영역 추출 네트워크(Region Proposal Network, RPN)를 추가한 것이 핵심이라고 할 수 있음. Faster R-CNN에서는 외부의 느린 선택적 탐색(CPU로 계산) 대신 내부의 빠른 RPN(GPU로 계산)을 사용함.

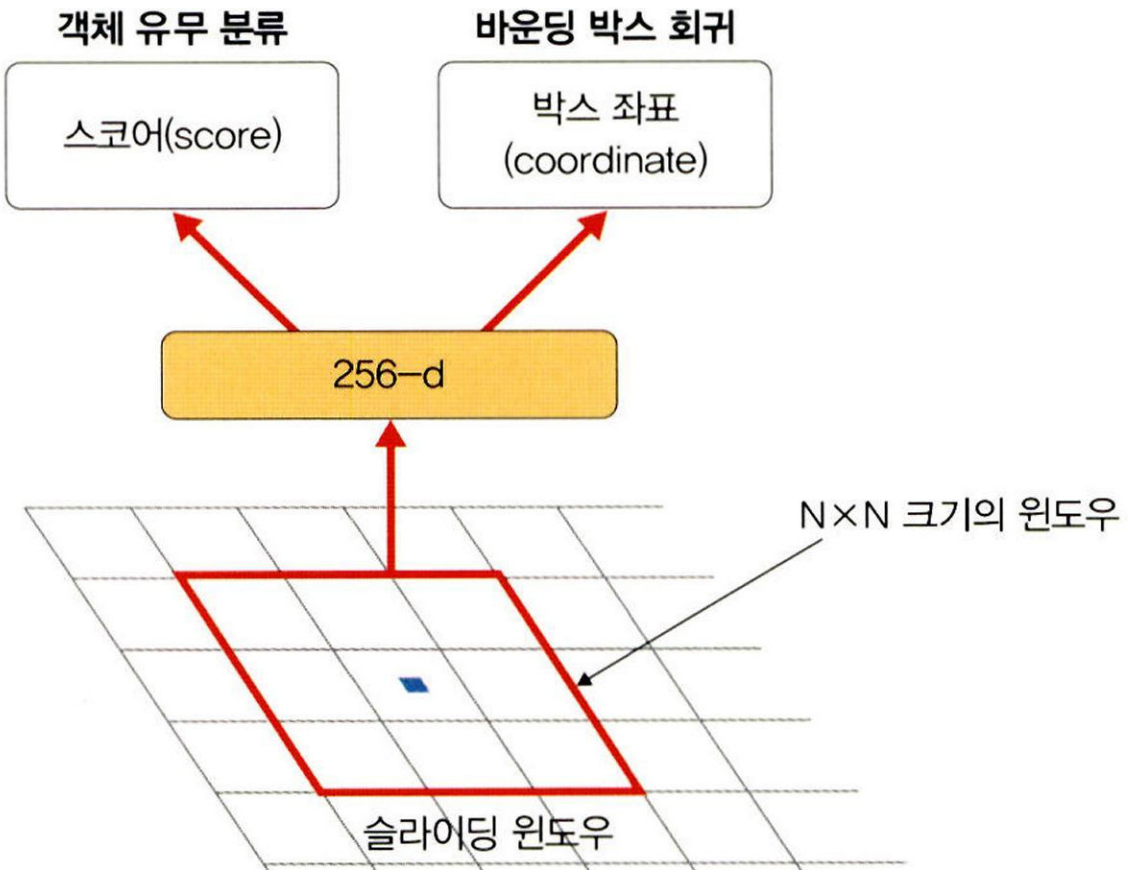
RPN은 다음 그림과 같이 마지막 합성곱층 다음에 위치하고, 그 뒤에 Fast R-CNN과 마찬가지로 RoI 풀링과 분류기(classifier), 바운딩 박스 회귀(bounding-box regression)가 위치함.

❯ 그림 6-43 Faster R-CNN



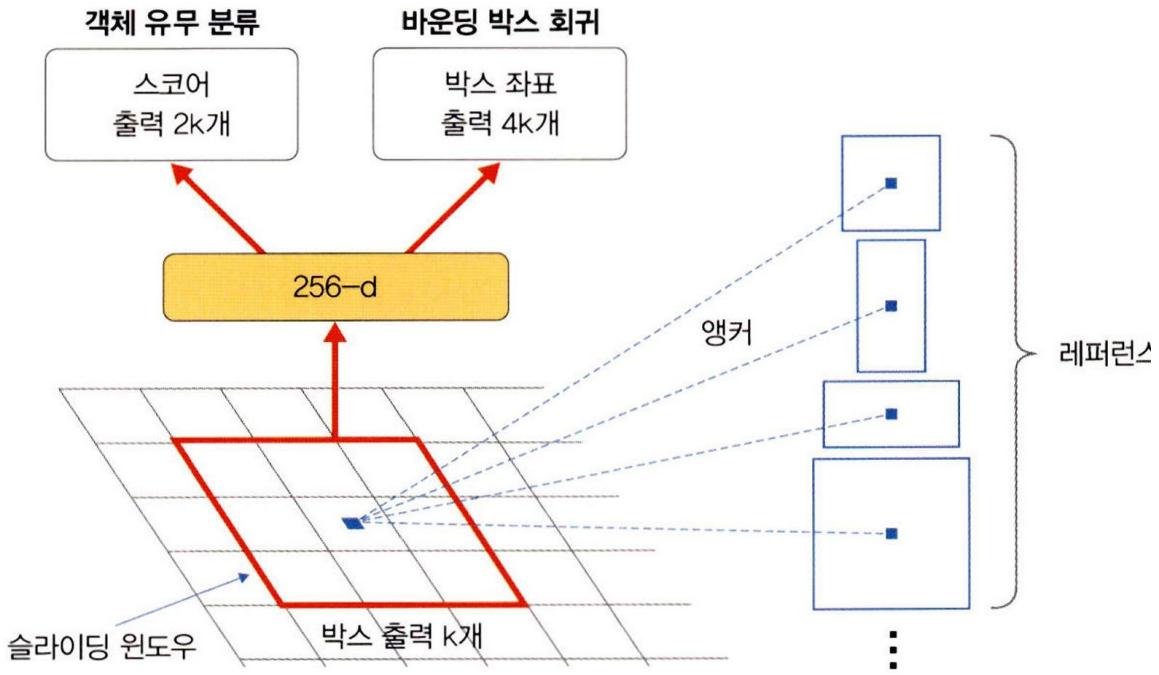
후보 영역 추출 네트워크는 특성 맵 N×N 크기의 작은 윈도우 영역을 입력으로 받고, 해당 영역에 객체의 존재 유무 판단을 위해 이진 분류(binary classification)를 수행하는 작은 네트워크를 생성함. R-CNN, Fast R-CNN에서 사용되었던 바운딩 박스 회귀 또한 위치 보정(좌표점 추론)을 위해 추가함. 또한, 하나의 특성 맵에서 모든 영역에 대한 객체의 존재 유무를 확인하기 위해서는 슬라이딩 윈도우 방식으로 앞서 설계한 작은 윈도우 영역(N×N 크기)을 이용하여 객체를 탐색함.

❯ 그림 6-44 후보 영역 추출 네트워크



하지만 후보 영역 추출 네트워크는 이미지에 존재하는 객체들의 크기와 비율이 다양하기 때문에 고정된 N×N 크기의 입력만으로 다양한 크기와 비율의 이미지를 수용하기 어려운 단점이 있음. 이러한 단점을 보완하기 위해 여러 크기와 비율의 레퍼런스 박스(reference box) k개를 미리 정의하고 각각의 슬라이딩 윈도우 위치마다 박스 k개를 출력하도록 설계하는데, 이 방식을 앵커(anchor)라고 함. 즉, 후보 영역 추출 네트워크의 출력 값은 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 2k개의 분류에 대한 출력과 x, y, w, h 위치 보정 값을 위한 4k개의 회귀 출력을 갖음. 예를 들어 특성 맵 크기가 w×h라면 하나의 특성 맵에 앵커가 총 w×h×k개 존재함.

❯ 그림 6-45 앵커



▼ 6.3 이미지 분할을 위한 신경망

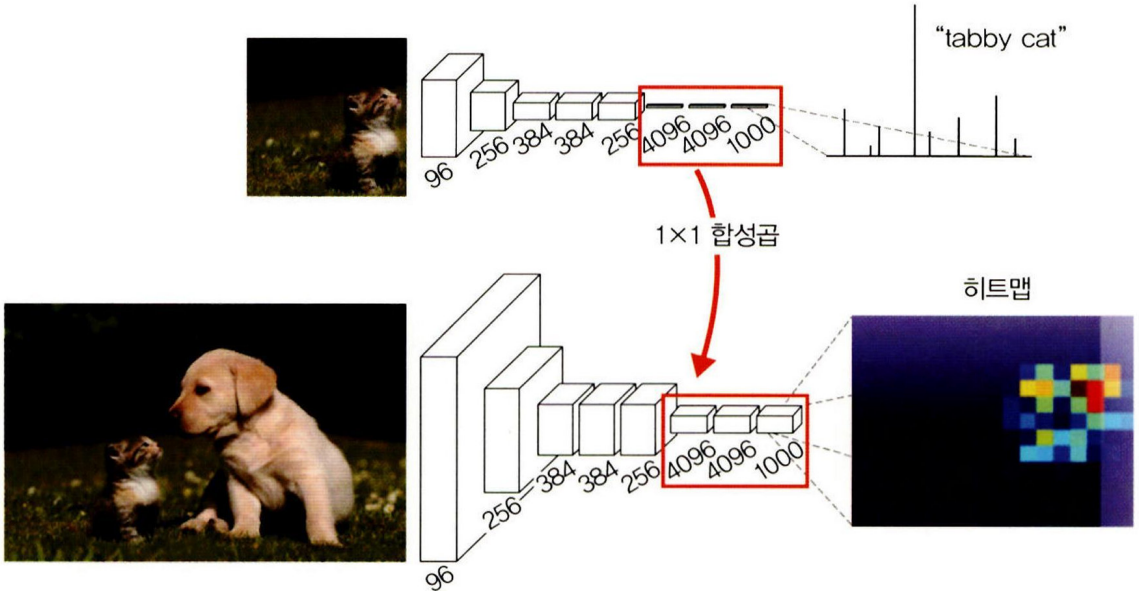
이미지 분할(image segmentation)은 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것임. 즉, 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출함. 이미지 분할의 대표적 네트워크는 완전 합성곱 네트워크, 합성곱 & 역합성곱 네트워크, U-Net, PSPNet, DeepLabv3/DeepLabv3+가 있음. 먼저 완전 합성곱 네트워크를 살펴보겠음.

6.3.1 완전 합성곱 네트워크

완전연결층의 한계는 고정된 크기의 입력만 받아들이며, 완전연결층을 거친 후에는 위치 정보가 사라진다는 것임. 이러한 문제를 해결하기 위해 완전연결층을 1×1 합성곱으로 대체하는 것이 완전 합성곱 네트워크임. 즉, 완전 합성곱 네트워크(Fully Convolutional Network, FCN)는 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크임.

예를 들어 다음 그림과 같이 AlexNet 아래쪽에서 사용되었던 완전연결층 세 개를 1×1 합성곱으로 변환하면 위치 정보가 남아 있기 때문에 히트맵(heatmap) 그림과 같이 고양이의 위치를 확인할 수 있음.

❯ 그림 6-46 완전 합성곱 네트워크



또한, 합성곱층으로 사용되기 때문에 입력 이미지에 대한 크기 제약이 사라지는 장점이 있음.

6.3.2 합성곱 & 역합성곱 네트워크

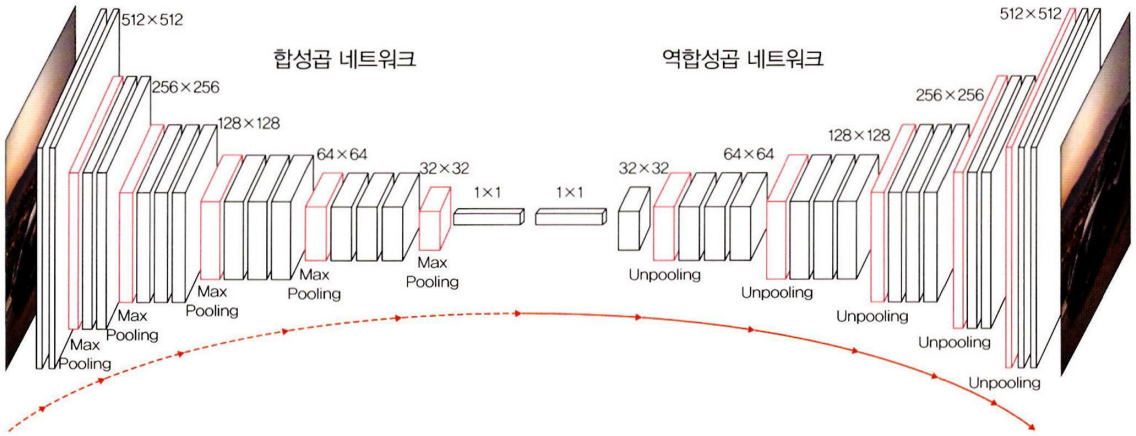
완전 합성곱 네트워크는 위치 정보가 보존된다는 장점에도 다음과 같은 단점이 있음.

- 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아짐.
- 낮아진 해상도를 복원하기 위해 업 샘플링 방식을 사용하기 때문에 이미지의 세부 정보들을 잃어버리는 문제가 발생함.

이러한 문제를 해결하기 위해 역합성곱 네트워크를 도입한 것이 합성곱 & 역합성곱 네트워크(convolutional & deconvolutional network)임.

역합성곱은 CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 사용함. 시멘틱 분할(semantic segmentation) 등에 활용할 수 있으며, 역합성곱을 업 샘플링(upsampling)이라고도 함.

❯ 그림 6-47 합성곱 & 역합성곱 네트워크



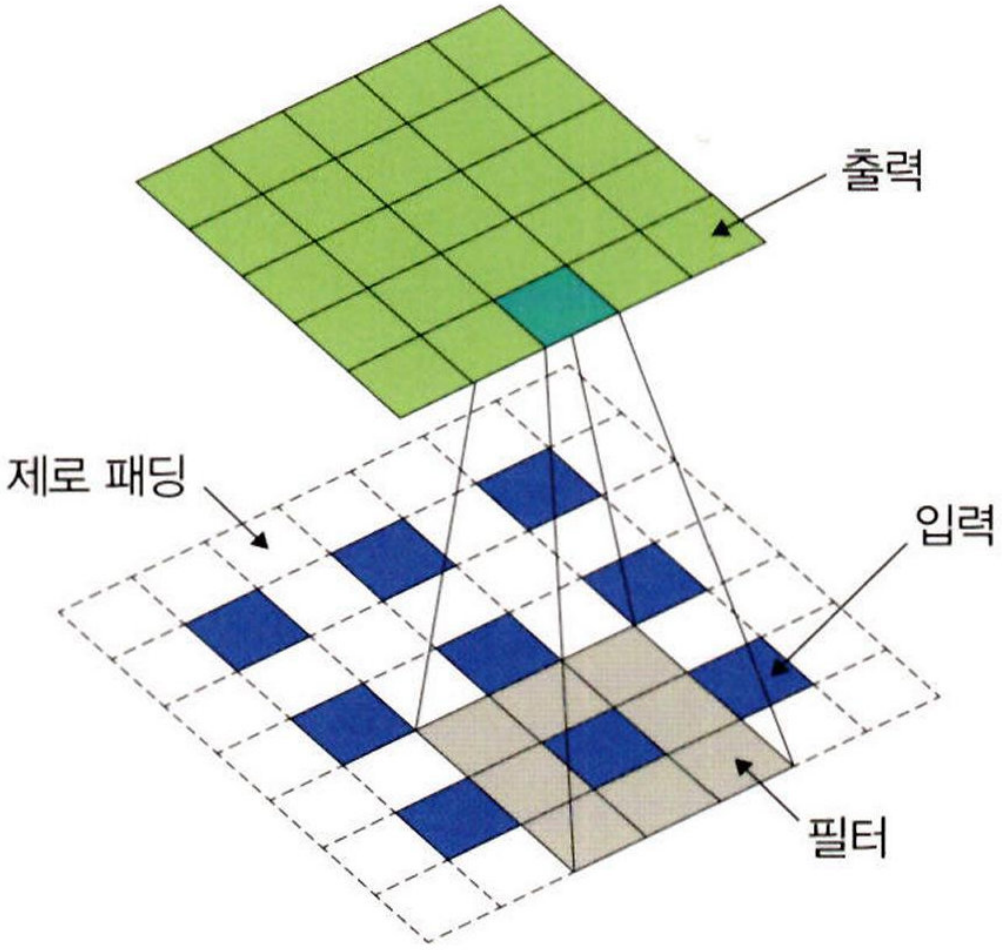
CNN에서 합성곱층은 합성곱을 사용하여 특성 맵 크기를 줄임. 하지만 역합성곱은 이와 반대로 특성 맵 크기를 증가시키는 방식으로 동작함.

역합성곱은 다음 방식으로 동작함.

1. 각각의 픽셀 주위에 제로 패딩(zero-padding)을 추가함.
2. 이렇게 패딩된 것에 합성곱 연산을 수행함.

오른쪽 그림에서 아래쪽의 파란색 픽셀이 입력이며, 초록색 픽셀이 출력임. 이 파란색 픽셀 주위로 흰색 제로 패딩을 수행하고, 회색 필터로 합성곱 연산을 수행하면 초록색이 출력됨.

❯ 그림 6-48 역합성곱 진행 방식



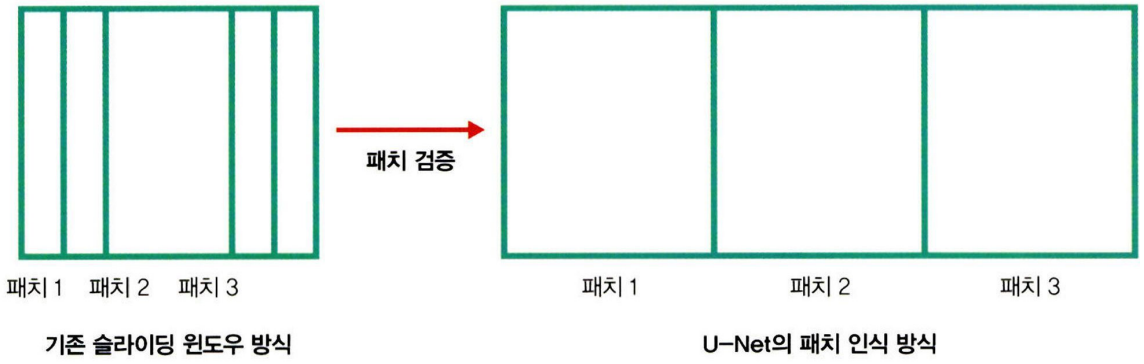
6.3.3 U-Net

U-Net은 바이오 메디컬 이미지 분할을 위한 합성곱 신경망임. 메디컬 이미지의 분할과 관련해서 항상 회자되는 네트워크가 U-Net임.

U-Net은 다음 특징이 있음.

- **속도가 빠르다:** 기존 슬라이딩 윈도우 방식은 이전 패치(patch)에서 검증이 끝난 부분을 다음 패치에서 또 검증하기 때문에 속도가 느렸음. 하지만 U-Net은 이미 검증이 끝난 패치는 건너뛰기 때문에 속도가 빠름.

❯ 그림 6-49 슬라이딩 윈도우 방식



기존 슬라이딩 윈도우 방식 U-Net의 패치 인식 방식

- **트레이드오프(trade-off)에 빠지지 않는다:** 일반적으로 패치 크기가 커진다면 넓은 범위의 이미지를 인식하는 데 뛰어나기 때문에 컨텍스트(context) 인식에 탁월함. 하지만 지역화에는 한계가 있음. 즉, 너무 넓은 범위를 한 번에 인식하기 때문에 지역화에는 약하기 마련인데, U-Net은 컨텍스트 인식과 지역화 트레이드오프 문제를 개선했음.

다음은 U-Net 구조임.

U-Net은 FCN을 기반으로 구축되었으며, 수축 경로(contracting path)와 확장 경로(expansive path)로 구성되어 있음.

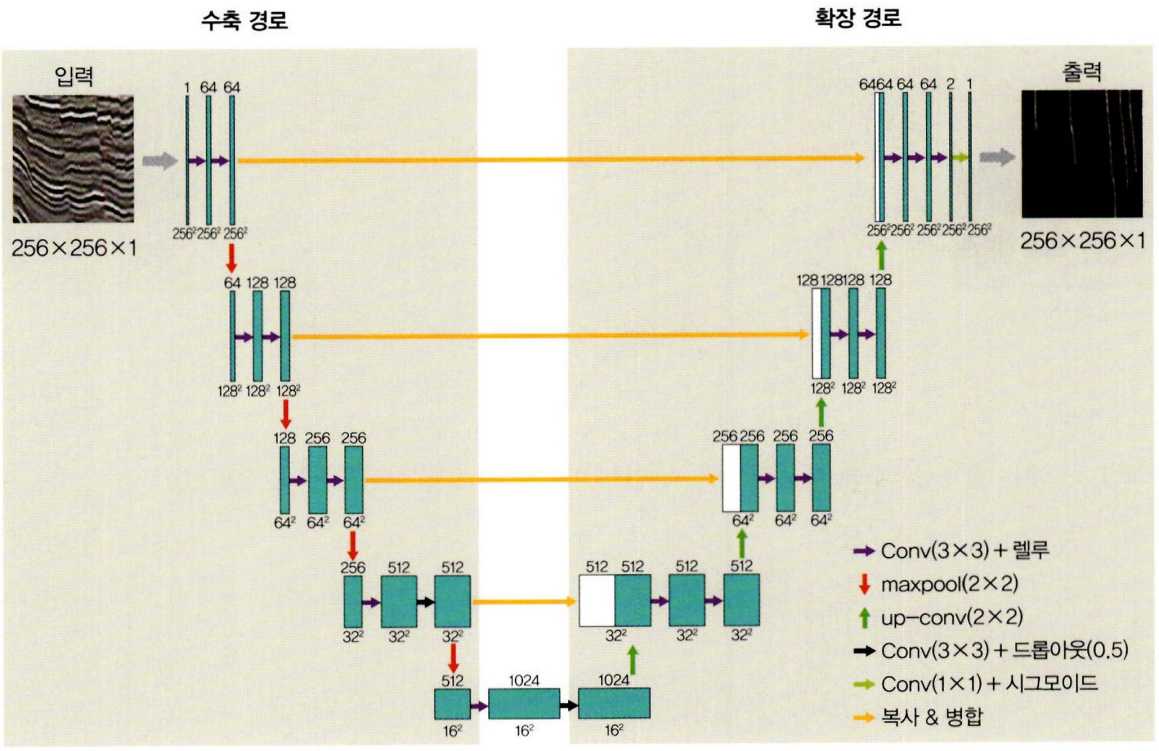
수축 경로는 컨텍스트를 포착하며, 확장 경로는 특성 맵을 업 샘플링하고 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합하여 정확한 지역화를 수행함.

U-Net은 3×3 합성곱이 주를 이루는데 각 합성곱 블록은 3×3 합성곱 두 개로 구성되어 있으며, 그 사이에 드롭아웃(dropout)이 있음. 다음 그림의 왼쪽 수축 경로에서의 블록은 3×3 합성곱 두 개로 구성된 것이 네 개가 있는 형태임. 그리고 각 블록은 최대 풀링(maxpool)을 이용하여 크기를 줄이면서 다음 블록으로 넘어감.

반면 다음 그림의 오른쪽 확장 경로에서는 합성곱 블록에 up-conv라는 것을 앞에 붙였음. 수축 과정에서 줄어든 크기를 다시 키워 가면서 합성곱 블록을 이용하는 형태임.

즉, 크기가 다양한 이미지의 객체를 분할하기 위해 크기가 다양한 특성 맵을 병합할 수 있도록 다운 샘플링과 업 샘플링을 순서대로 반복하는 구조로 되어 있음.

그림 6-51 U-Net



6.3.4 PSPNet

PSPNet(Pyramid Scene Parsing Network)은 CVPR(The IEEE Conference on Computer Vision and Pattern Recognition) 2017에서 발표된 시멘틱 분할 알고리즘임.

PSPNet 역시 완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈을 추가했으며 훈련 과정은 다음과 같음.

1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링을 함. 즉, 1×1, 2×2, 3×3, 6×6 크기로 풀링을 수행하는데, 이때 1×1 크기의 특성 맵은 가장 광범위한 정보를 담음. 각각 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담는다고 이해하면 됨.
2. 이후 1×1 합성곱을 사용하여 채널 수를 조정함. 풀링층 개수를 N이라고 할 때 출력 채널 수=입력 채널 수/N이 됨.
3. 이후 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링함. 이 과정에서 양선형 보간법(bilinear interpolation)이 사용됨.
4. 원래의 특성 맵과 1~3 과정에서 생성한 새로운 특성 맵들을 병합함.

그림 6-52 PSPNet

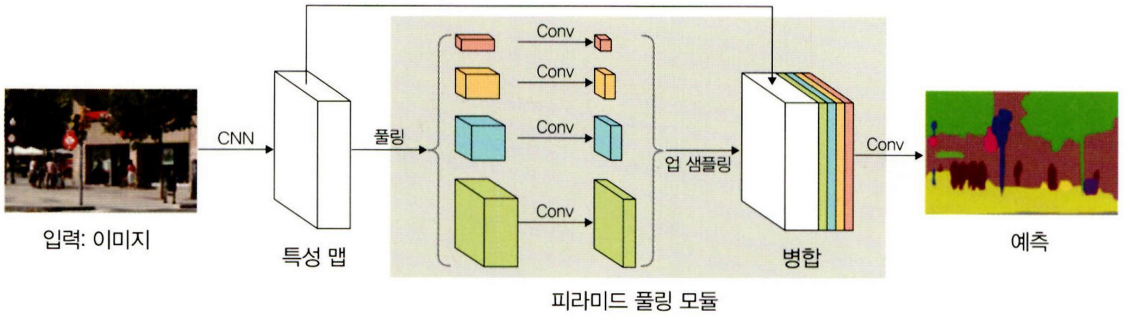
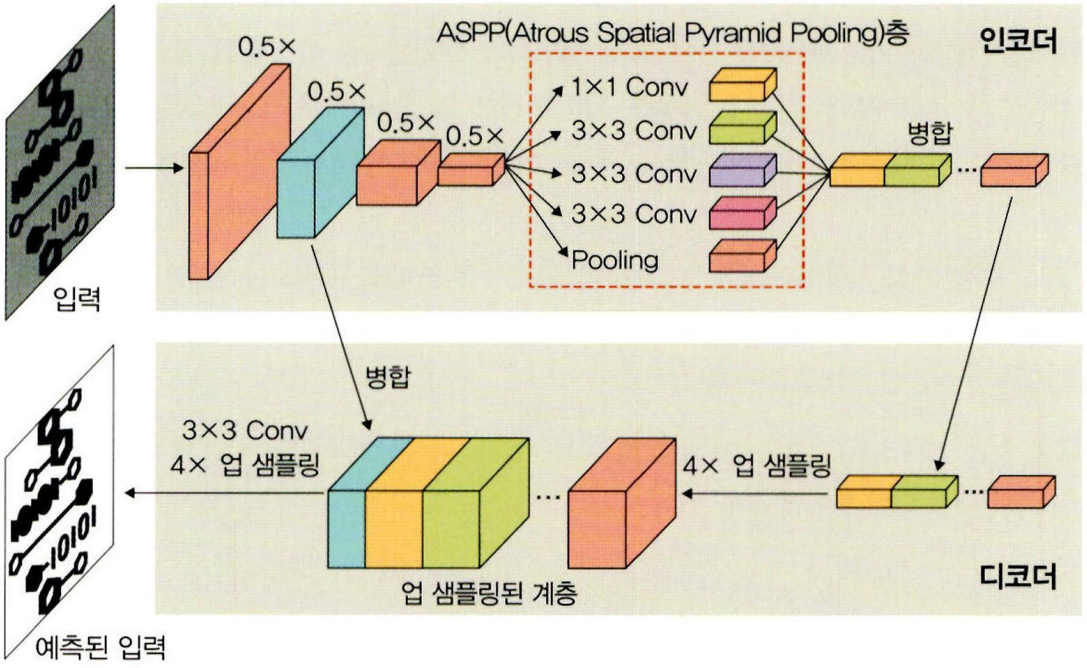


그림 6-52는 풀링을 네 개 사용했지만, 구현에 따라서 다르게 설정할 수 있음.

6.3.5 DeepLabv3/DeepLabv3+

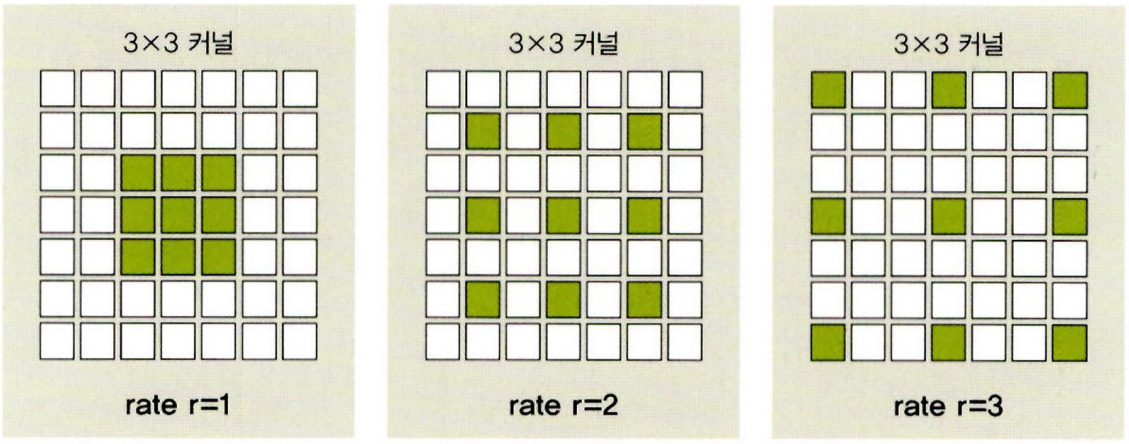
DeepLabv3/DeepLabv3+ 역시 완전연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용하는 네트워크임. 인코더과 디코더 구조를 가지며, 일반적으로 인코더-디코더 구조에서는 불가능했던 인코더에서 추출된 특성 맵의 해상도를 Atrous 합성곱을 도입하여 제어할 수 있도록 했음.

그림 6-54 DeepLab의 인코더-디코더 구조



Atrous 합성곱은 다음 그림과 같이 필터 내부에 빈 공간을 둔 채로 작동함. 얼마나 많은 빈 공간을 가질지 결정하는 파라미터로 rate가 있음. rate r=1일 경우 기존 합성곱과 동일하게 빈 공간을 가지며, r이 커질수록 빈 공간은 더 많아짐.

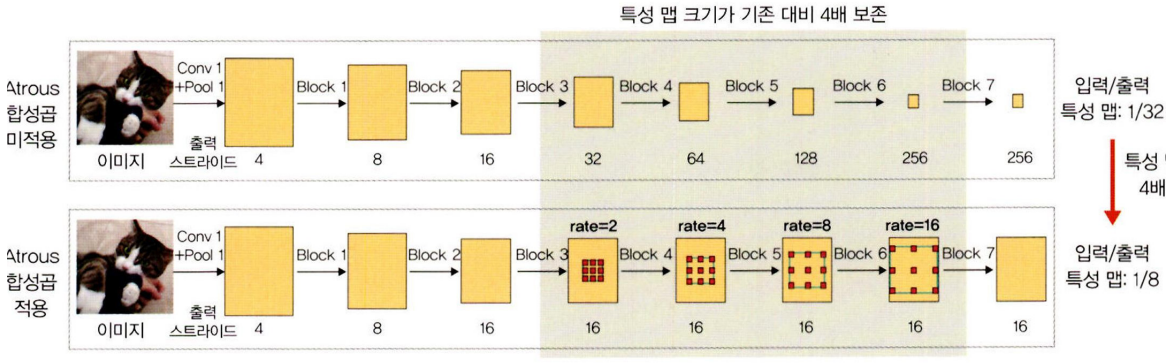
❯ 그림 6-55 Atrous 합성곱



보통 이미지 분할에서 높은 성능을 내려면 수용 영역(receptive field)의 크기가 중요한데, 수용 영역을 확대하여 특성을 찾는 범위를 넓게 해 주기 때문임. Atrous 합성곱을 활용하면 파라미터 수를 늘리지 않으면서도 수용 영역을 크게 키울 수 있기 때문에 이미지 분할 분야에서 많이 사용함.

즉, 다음 그림과 같이 일반적인 CNN을 적용하면 출력은 입력에 비해 1/32로 줄어들지만, Atrous 합성곱을 적용하면 1/8로 줄어듦. 따라서 특성 맵 크기가 기존 대비 4배 보존된 것을 확인할 수 있음.

❯ 그림 6-56 Atrous 합성곱 효과



5~6장에 걸쳐 합성곱 신경망을 자세히 알아보았음. 이어서 7장에서는 시계열 분석을 살펴보겠음.